

California State University Long Beach
Computer Engineering and Computer Science Department
CECS 440 – Computer Architecture
Lab 5 – ALU and Register File

Objectives:

- Model a portion of the CPU datapath in structural Verilog: an ALU, a register file (RF), and an ALU output register. The datapath reads two operands from the RF, performs an ALU operation, and stores the result in the dedicated output register.

Background:

This lab assignment is designed to help you strengthen your understanding of MIPS instructions and support the construction of the MIPS datapath. This lab focuses on supporting the R-format and I-format of arithmetic and logic instructions. Please review the relevant materials beforehand to prepare for this assignment.

Assignment:

Your main task in this assignment is to interconnect the functional blocks to create the system shown in Figure 2.1. This figure illustrates the top-level view of the design, which consists of an ALU, a register file, and an ALU output register. The overall design will be implemented in **design.v**, which serves as the top-level design module.

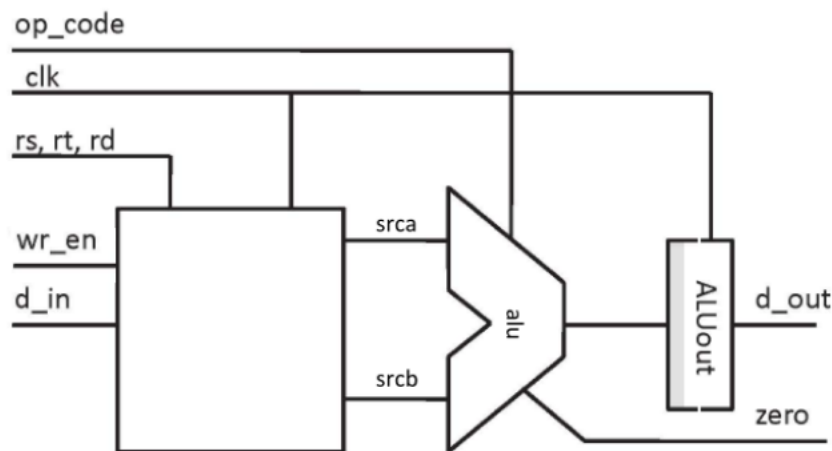


Figure 2.1 Block Diagram of ALU and RF

Steps:

1. Install Vivado: Install Vivado on your laptop or desktop. Any version is acceptable. In the labs, we will showcase Vivado 2019.2.
2. Create a Folder Structure for CECS 440 Labs. If your folder is on the local drive, back up your work frequently (onto OneDrive). Paths on OneDrive may contain **spaces** or may be **too long** for Vivado.

Examples:

Local drive examples:

> This PC > Local Disk (C:) > CECS_440

Or

> This PC > Local Disk (C:) > Users > Public >

If the file path is not too long, create a folder for **lab5** in the **labs** folder.

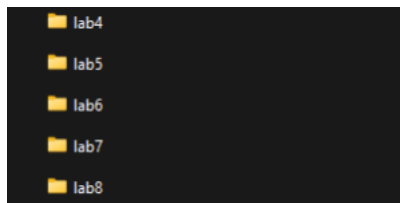
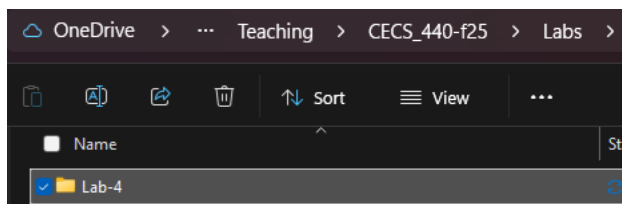


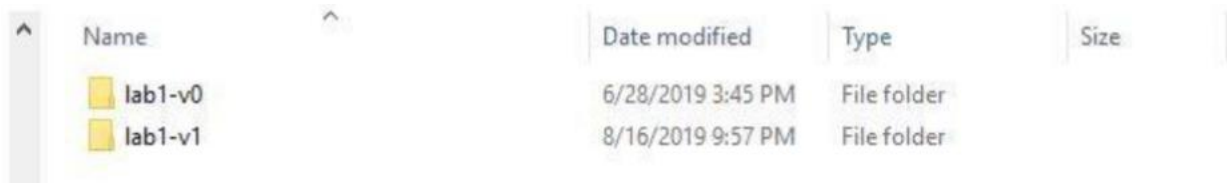
Figure 1. Folder structure (this project is lab 5, create folder accordingly)

You might be able to store your labs directly on OneDrive. However, the path to OneDrive might have spaces or might be too long. You should use this location for backup.



3. Start Vivado 2019.2 and follow the instructions on how to create a project (Version 2019.2 on SVL; the version on your computer may be different)

- a. Name the project lab5-v0. Later, if needed, you may create additional versions such as lab5-v1, lab5-v2, etc. After creating the project, the file structure should look as shown below.



Name	Date modified	Type	Size
lab1-v0	6/28/2019 3:45 PM	File folder	
lab1-v1	8/16/2019 9:57 PM	File folder	

Figure 2. Folders after creating two versions of the project (you may need only one, or more)

4. Add **alu** design source from lab1 or create a new design source and name it **alu.v**. Copy the **alu** source code from lab 1 into a file named **alu.v** in this lab5 project.
5. Create design sources for the **ALUout** register (aluout.v) and **register file** (regfile.v). It is important to note that *register file* should *write on a negative edge*. Write the code as given in Figures 2.2 and 2.3.

Note: we3 is write_enable, ra1 is source 1 address, ra2 is source 2/destination address, .wa3 is destination, wd3 is data input, .rd1 is data obtained from source 1, and rd2 is data obtained from source 2.

```
// Code your design here
`timescale 1ns/100ps

//ff with load enable
module aluout #(parameter width = 8)
    ( input clk, reset, load,
      input [width-1:0] d,
      output reg [width-1:0] q);

    always @(.)
    begin
        if(reset) ;
        else if(load) ;
    end
endmodule
```

Figure 2.2 Verilog code for ALUOut register (aluout.v)

```
`timescale 1ns/100ps

module regfile #(parameter width = 8)
( input clk,
  input wr_en,      // write enable
  input [4:0] rd_addr1, rd_addr2, wr_addr, // rs, rt, rd
  input [width-1:0] wr_data, // d_in = data in
  output [width-1:0] rd_data1, rd_data2); // srca, srcb

  reg [31:0] rf[31:0];
  // three ported register file
  // read two ports combinationaly
  // write third port on rising edge of clk
  // register 0 hardwired to 0
  // note: for pipelined processor, write third port
  // on falling edge of clk
  always @(negedge clk)
    if (wr_en) rf[wr_addr] <= wr_data;

  assign rd_data1=(rd_addr1 !=0) ? rf[rd_addr1] : 0;
  assign rd_data2=(rd_addr2 !=0) ? rf[rd_addr2] : 0;
endmodule
```

Figure 2.3 Verilog code for register file (regfile.v)

6. Create a design source named **design.v**. You will need to instantiate **regfile** and the **alu** according to Figure 2.1 (that is: fill in the port list for those two modules). Make sure that the design.v file is in your sources file path of your project or else it will not be able to access your Verilog files that you create. Refer to Fig 2.4.
 - a. You may wish to add a test bench to test your register file. We do not require it, but it will help you achieve proper functionality.
 - b. Note: For ALUOut, set load to 1 in module instantiation.

```

22  `include "regfile.v"
23  `include "aluout.v"
24  `include "alu.v"
25
26  `define datasize 32
27  module simple_datapath(
28      input [2:0]    op_code,
29      input          clk, reset,
30      input [4:0]    rs, rt, rd,
31      input          wr_en,
32      input [`datasize - 1:0] d_in,
33      output [`datasize - 1:0] d_out,
34      output c, n, z, p
35  );
36
37      wire [`datasize-1:0] srca, srcb, aluout;
38      // rfDataOut1, rfDataOut2,
39
40      regfile #(`datasize) rf (    ); //fill in the port list
41      alu #(`datasize) al (    ); //fill in the port list
42      aluout #(`datasize) ALUout (    );
43      assign d_out = aluout;
44  endmodule

```

Figure 2.4 Verilog code for design.v

- Finally, create **testbench.v**. The skeleton code is provided as a text file on Canvas. Refer to **Figures 2.5** and **2.6** for examples of console output and simulation results. In the testbench, between **lines 42 and 88**, you will find assignment statements for test cases—these require you to fill in the values on the right-hand side. **Use Table 1** to help complete the assignment statements. *****It is very important that you understand how to explain these test cases*****

Table 1:

Test Case	Operation	op_code	rs	rt
1	add \$ALUout, \$3, \$4			
2	inc \$ALUout, \$3			
3	and \$ALUout, \$1, \$2			

4	or \$ALUout, \$1, \$2			
5	xor \$ALUout, \$2, \$1			
6	not \$ALUout, \$2			
7	sl \$ALUout, \$4			
8	nop			

```

Checking test case
srca =          500, srcb =          1000
add $ALUout, $ 3, $ 4 performed
sum =          1500
test passed

```

Figure 2.5 Sample console output

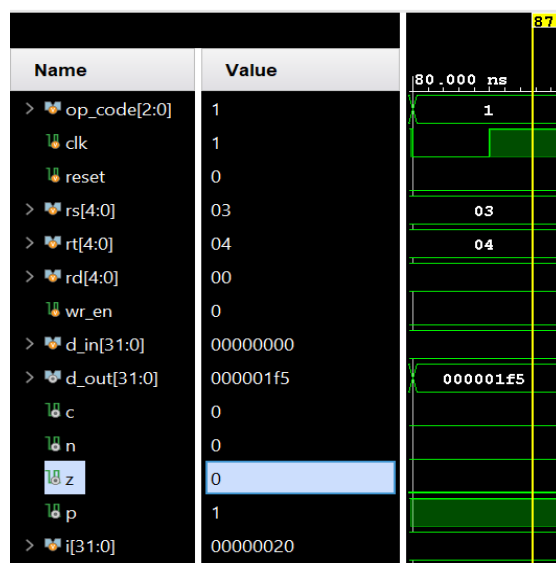


Figure 2.6: Sample Simulation Output

- Each operation is an instruction where the **destination register is *always* \$ALUout and does not need to be specified**, rs is the number of the **first source register**, rt is the number of the *optional* **second source register (for operations that take 2 source register operands)**.

Submission:

Once you have verified the proper functionality of your project, copy the contents of **alu.v**, **aluout.v**, **regfile.v**, **design.v**, and **testbench.v** to a text file in that order, named **lab5.txt**, and upload it to the Canvas for Lab 5. Additionally, upload your Lab report (see the LabReportTemplate in the Documents folder on Canvas), and include populated **Table 1** in your lab report. Write your report using the **Lab Report Template** available on Canvas. Be sure to include:

- Screenshots of your code (for test bench, include only the first few lines that show instantiation of the design module)
- Three random Console output s
- Three random waveform screenshots showing a three selected test cases

Note: Keep all lab files, as they will be needed for future labs.

Created by Josh Hayter, Jelena Trajkovic and Bronson Garel (Copyright © 2025)